Policy Engine Services Containerization Design

- Version History
- Related Document(s)
- Overview
- Service containerization
  - Santorini
  - Non-Santorini
- Microservices
  - policy-catalog
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Central Interpreter
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Execution-planner
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Scheduler
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Job-creator
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Job-dispatcher
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability
    - Security
    - Refactor checklist
  - Job-executor
    - Resource Ownership
    - Containerization
    - Deployment
    - NDU
    - HA
    - Manageability
    - Serviceability

        - Security
        - Refactor checklist
    - Activity

Version History

Rev	Date	Author	Description
1	14 Apr 2023	Zhu, Mike	Initial version for policy engine micro-service containerization design
2	04 May 2023	Zhang, Raynor	Update policy-catalog to conform with the refined check-list
3	10 May 2023	Zhu, Mike	Add design for scheduler service
4	10 May 2023	Yan, JinRu	Add design for execution planner
5	10 May 2023	Chang, Shaofeng	Add design for central interpreter
6	10 May 2023	Hu, Xiaolei	Add design for job creator/job-dispatcher
7	10 May 2023	Chen Li	Add design for job executor

Related Document(s)

Policy Engine Subsystem

[HLD] Micro Service and Micro Service Shell Specification

Considerations for Schema Patch and Data Migration

Log Persistency and Log Bundle Creation in Santorini

Santorini Logging Guidelines

Overview

This document depicts and consolidates the non-function aspects of policy engine micro-services in different PPDM form factors including Santorini and non-Santorini (PPDM standalone, DM5500).

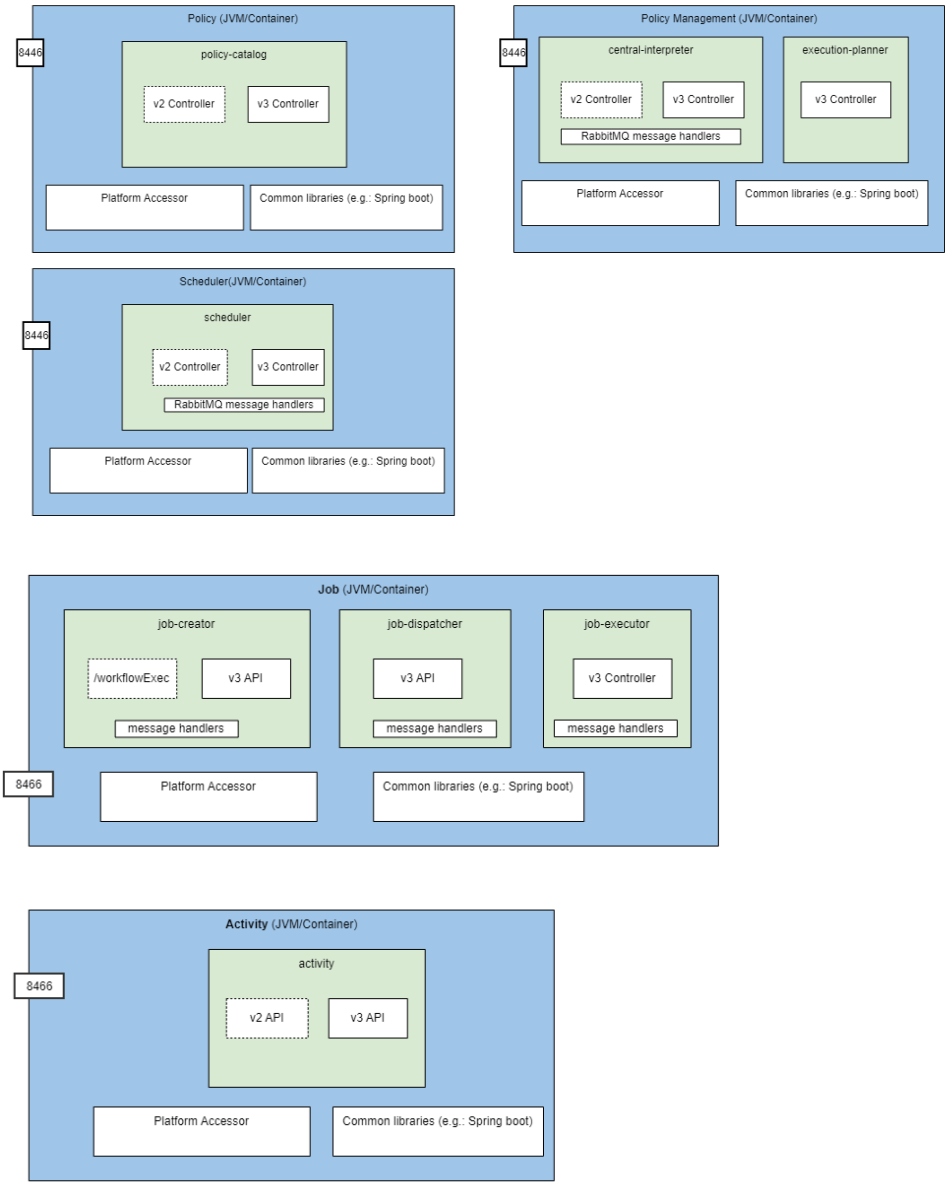
This document covers the requests for below two EPICs.

PPDM-225193 - Getting issue details...	STATUS
PPDM-224376 - Getting issue details...	STATUS

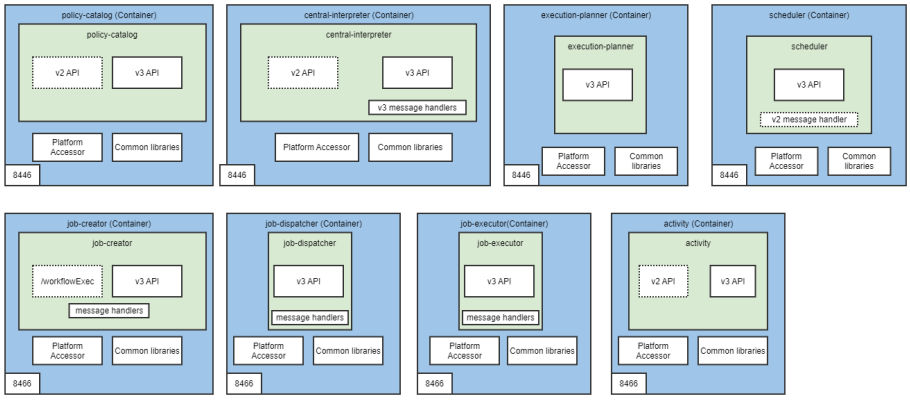
Service containerization

Santorini

For Santorini scale-out version in Santorini Wave 1. x, policy engine micro-services would still run in the data protection appliance. Resource and performance is still major factor to consider, policy engine services would be deployed as below layout.



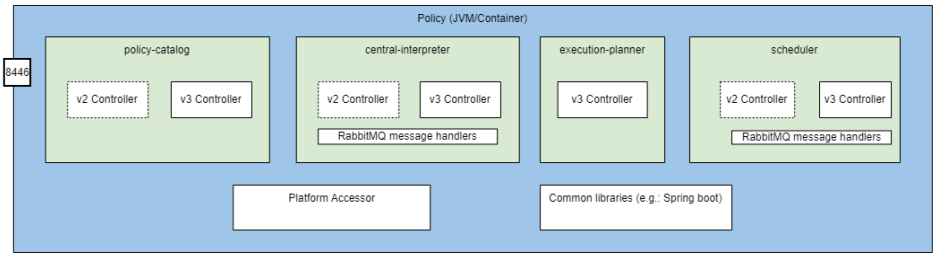
For Santorini hyper-scale version, policy engine micro-services would be deployed as per micro-service containerization.

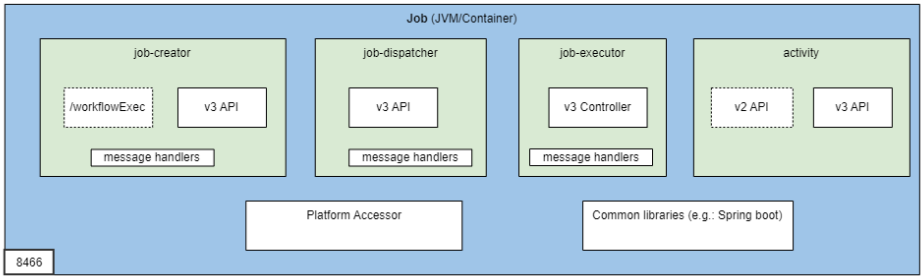


Non-Santorini

PPDM non-Santorini form factors include PPDM standalone, and DM appliance (DM5500).

Policy engine micro-services would be grouped into two containers in PPDM standalone and DM5500 to achieve better performance (reduce the network latency) and low resource (majorly memory) footprint.





Microservices

policy-catalog

Resource Ownership

policy-catalog service is the owner of the "policy" resource. It provides REST APIs for the user to do management of their Policy and emit notifications to the subscribers.

API Resources Ownership

API Type	Resource	Supported Operations
REST API	/v3/policies	CRUDQ
	v3/protection-policy-settings	RUQ

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables
ES	index_policy	index_protection_life_cycle
	index_policy_setting	index_setting

Containerization

For non-Santorini PPDM form factors, the policy catalog will run in the same container with the services: central interpreter, execution planner, and scheduler.

For Santorini scale-out, policy-catalog service runs in its owner container considering the below reasons:

1. Not co-locate with other micro-services with more complicated service dependencies for NDU since policy-catalog does not depend on other PPDM services, only depends on infrastructure services: elastic search, rabbitMQ, CIAM, Audit log, and Alert
2. and can be deployed as multiple pods to support HA in the future version

API SDK

API SDK will be generated from API schema (defined by YAML) for APIs provided by the policy-catalog service.

policy-catalog API consumer shall adopt policy-catalog API SDK to interact with policy-catalog service

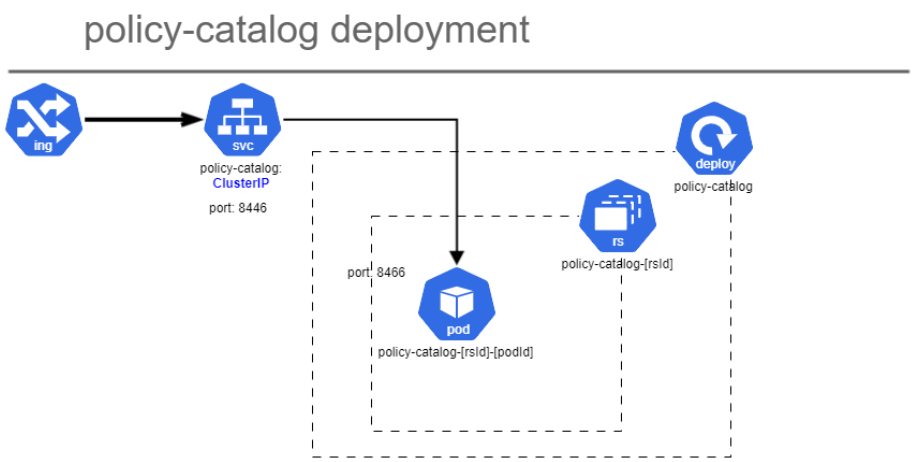
policy-catalog service shall adopt the API SDK of its dependent service, such as CIAM, Audit, Metrics, etc.

Deployment

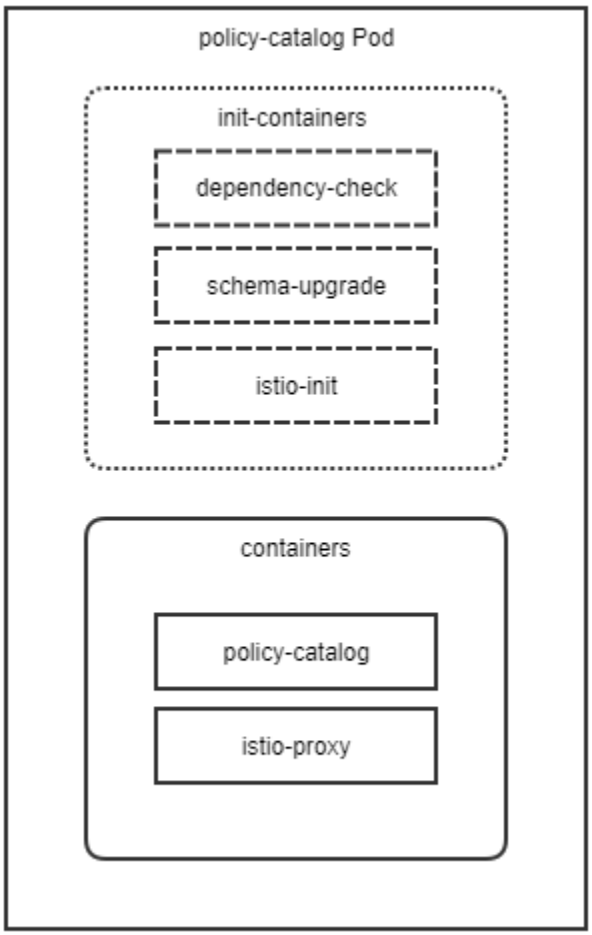
policy catalog would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

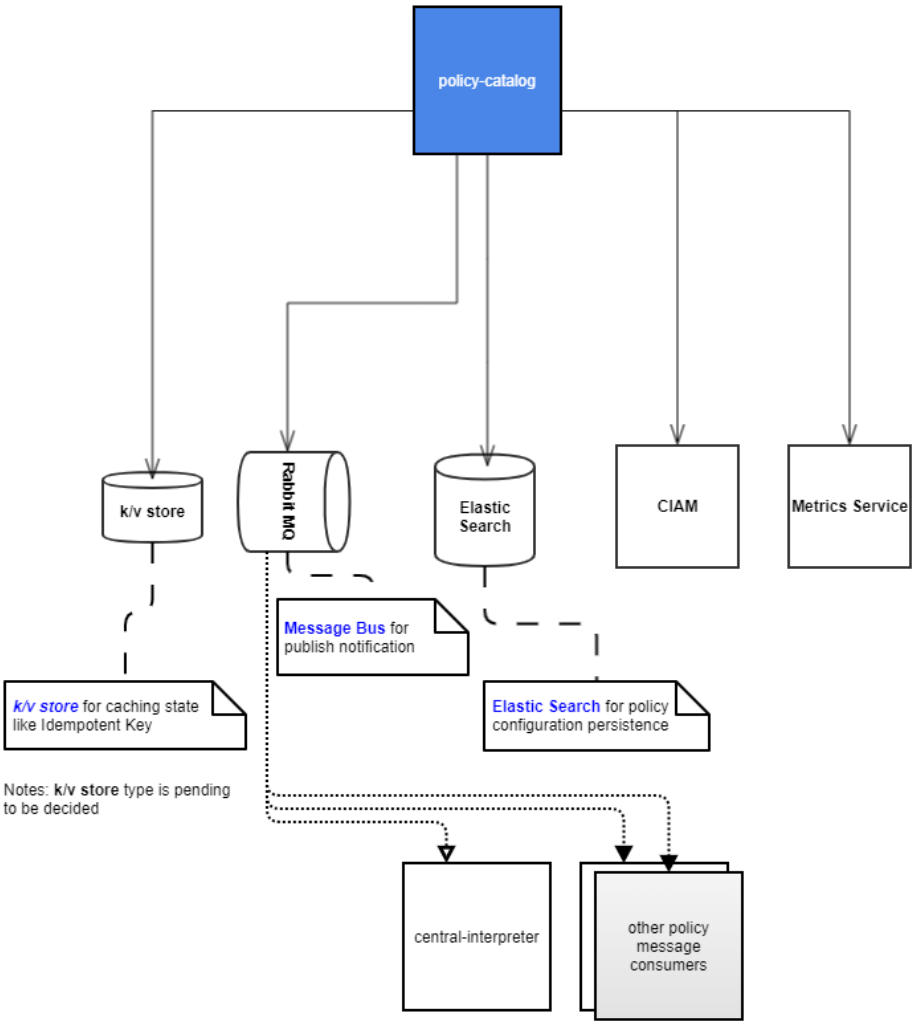
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.
Storage (App)	**Infra** / **Estimated Size** ES: 1024 * 5k *48 = 240Mi * 2 = 500Mi • 1024: max policy number per PPDM system • 5k: stage size with 16 schedules/retentions • 48: maximum stage size in a policy PSQL: N/A		Zhang, Raynor - Should the Idempotent API key-value cache be counted in this part? YES: we shall estimate the potential API calls per idempotent persistent time to get the estimated storage usage.
Storage (Log)	100	<TODO>	in MB/day

NDU

policy-catalog service is designed as

Stateless

no local service state per Pod

Versioned API

policy.yaml

Versioned messages

Policy v3 Notifications

(pending on the common design (Zhang, Raynor) of how rabbitMQ message will be versioned)

API backward/forward compatibility

- As a consumer service (API provider) policy API supports backward compatibility
- For client service calling into policy API
    - Policy SDK-based Internal client service
        - For client service with older version SDK, no problem since API supports backward compatibility
        - For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade Architecture Spec#APIs
    - Policy API direct client service
        - For client service using older version API schema, no problem since API supports backward compatibility
        - For client service using a newer version API schema Same as SDK-based #2
- policy-catalog is a client service of CIAM, Audit log, and Metric service, it shall integrate SDK of CIAM, Audit log, and Metrics services. CIAM, Audit log, and Metrics service shall be upgraded before the policy-catalog service

Fully data schema ownership

- index_policy
- index_policy_settings
- Adopting Micro Schema Manager for schema patch and data migration

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

schema patch: within 10 seconds for patching policy data part (max: 1024 policies), not counting other overhead to launch schema patch init-container.

Long Run job handling

policy-catalog service does not support any long-run job that may run across an upgrade, so it does not impact any job runs before the upgrade.

For any cache e.g.: Idempotent, performance shall be externally persisted which shall not impact the functionality.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search, CIAM

Soft dependencies

Central-Interpreter, Alert, Audit

Dependency diagram

Outgoing Dependencies

HA

policy-catalog service will be deployed as 1 instance in Santorini Wave 1.0 but would be deployed as multiple instances post Wave 1.0 since the policy-catalog service is designed as stateless. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period. Rely on K8S HA to restart service instances when HA events happen.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
policy-catalog	<= 30 seconds	******** Started Component policy-catalog in 26.613 seconds (JVM running for 26.694) ********
MSU (Micro Schema Upgrade)	<= 3 seconds	per the data migration patch, data migration chance shall be restricted MSU container may take different periods per the data to be migrated, the maximum policy configured per system is 1024, and 1 data migration for up to 1024 shall be completed within **3 seconds**; while the other overhead of MSU init-container should be out of control of policy-catalog schema patch. 2023-04-11T05:28:54.29942114-04:00 stdout F 09:28:54.295 [main] INFO c.e.d.e.e.s.m.MicroSchemaUpgradeManager - Micro schema manager started... ... 2023-04-11T05:29:01.537547027-04:00 stdout F 09:29:01.537 [main] INFO c.e.d.e.e.s.m.MicroSchemaUpgrader - policy-service Schema upgrade started ... ... 2023-04-11T05:29:02.473489162-04:00 stdout F 09:29:02.472 [main] INFO c.e.d.e.e.s.m.MicroSchemaUpgrader - policy-service Schema upgrade succeeded ... 2023-04-11T05:29:02.475980053-04:00 stdout F 09:29:02.475 [main] INFO c.e.d.e.e.s.m.MicroSchemaUpgradeManager - Micro schema manager ended...
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the policy-catalog service

Shutdown time

Item	Graceful shutdown time	Comments
policy-catalog	<= 3 seconds	2023-05-03T20:49:45.669984371-04:00 stdout F 2023-05-04T00:49:45.669Z INFO [] [Thread-4] [][][][][] [c.e.d.p.p.r. PolicyServiceApplication.protectedShutdown(462)] - Shutting down... ... 2023-05-03T20:49:46.877355377-04:00 stdout F 2023-05-04T00:49:46.876Z INFO [] [Thread-4] [][][][][] [o.s.a.r.l. SimpleMessageListenerContainer.lambda$shutdownAndWaitOrCallback$3(656)] - Successfully waited for workers to finish.
Others	TBD	other overhead of Pod shutdown is out of the control of the policy-catalog service

Idempotent API

Adopt the common Idempotent API framework to deliver Idempotent API

Advanced resilience improvement in API Idempotent framework such as how to persist the state in client/server-side are tracked by

PPDM-225171 - Getting issue details... STATUS

Data consistency (service state persistence)

Policy catalog API itself is a very quick operation that includes the below steps, no state persistence is needed for this API. Since there are three steps during policy CUD, and the time duration between these steps should be very short, the ES and MQ are basic infra services that are deployed as HA with multiple instances, and ES/MQ client also supports retry, the probability to have inconsistent data during policy CUD is very low.

1. Needs access ***ElasticSearch*** to read/write protection policy configuration
2. Broadcast resource CUD notification to ***RabbitMQ*** Message Bus
3. Create audit logs when CUD policy configuration happens

The result of *step 2* is very critical because that would impact the protection execution plan, which may result in Data Unavailable. Policy service shall make sure the message can be delivered at least once. Two items:

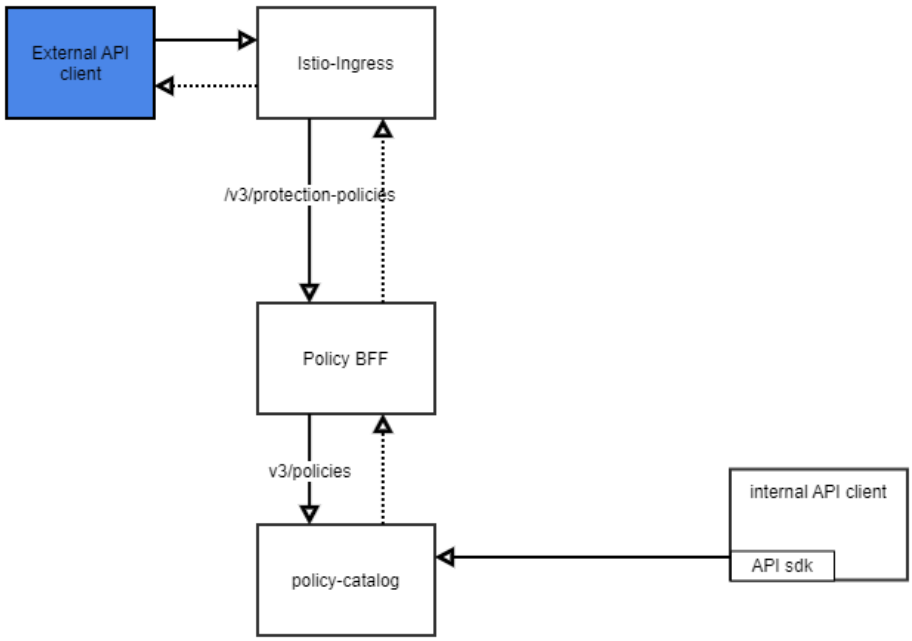
1. Configure the message delivery retry period to fit with the RabbitMQ auto restart time, retry period shall be >= RabbitMQ auto restart time.
2. Cache (need persist) the message that is not delivered successfully and re-delivery it when rabbitMQ is back to work.

The additional changes would be tracked by **PPDM-245611** - Getting issue details... STATUS .

Refer to Data integrity for more details.

Manageability

policy-catalog service provides the Rest API for CRUD policy catalogs, and basic API validations will be done by the policy-catalog service itself, other advanced cross-domain validations would be split into policy BFF/management service (central-interpreter) to reduce service external dependencies to support NDU.



Serviceability

Logging

policy-catalog service will follow the common logging framework to produce the service logs.

The policy-catalog service will adopt 2.1. Write to local files and use a sidecar before metrics are sent over Instrument SDK. Will further evaluate whether switching to 2.3. Write to STDOUT and use Fluentbit

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Spike: Policy Service Observability Design

Security

PPDM RBAC

policy catalog CRUD operation is enforced with the below privileges.

Operation	Privileges	
Create	MANAGE_POLICIES	
Update	MANAGE_POLICIES	
Delete	MANAGE_POLICIES	
GetById	VIEW_POLICIES	
Query	VIEW_POLICIES	

Refactor checklist

PPDM Service Refactor Checklist -Policy Catalog

Central Interpreter

Resource Ownership

API Resources Ownership

API Type	Resource	Supported Operations	Comments
REST API	v3/protections no resource, just operation	C	initiate 'Backup Now", "Replicate Now" functions, aka 'Adhoc Backup or Replicate'
REST API	v3/protection-policies	CRUDQ	protection policy v3 version, backed up by /v3/policies.
REST API	v3/protection-policy-summaries	RQ	
REST API	v3/protection-policies-batch	U	Batch patch to update policy state to enable/disable policy
REST API	v3/protection-details	RQ	
REST API	v3/protection-metrics	RQ	

Containerization

For non-Santorini PPDM form factors, the central-interpreter will run in the same container with the services: policy-catalog, execution planner, and scheduler.

For Santorini scale-out, central-interpreter service runs in its owner container considering the below reasons:

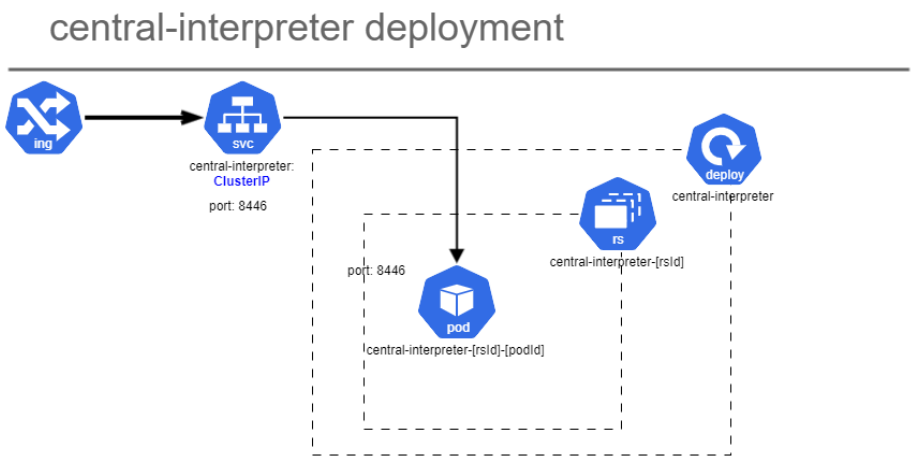
1. Not co-locate with other micro-services with more complicated service dependencies for NDU since central interpreter does not depend on other PPDM services, only depends on infrastructure services: elastic search, rabbitMQ, CIAM, Audit log, and Alert
2. and can be deployed as multiple pods to support HA in the future version

Deployment

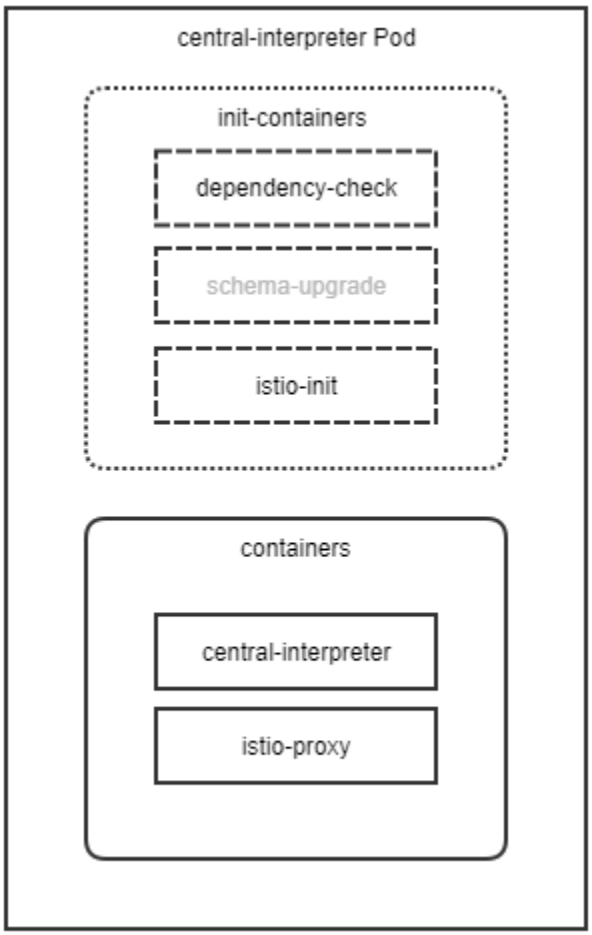
central-interpreter would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

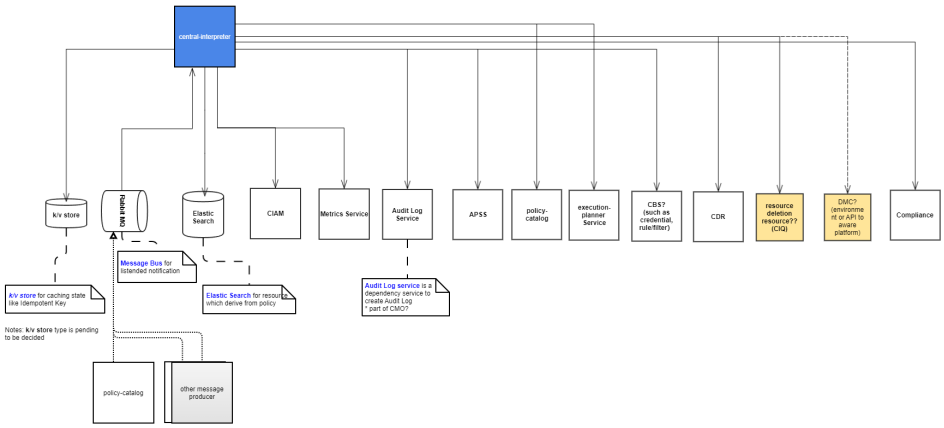
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	256 MB	512 MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.
Storage (App)			
Storage (Log)	100	<TODO>	Most of the time , the central-interpreter works based on adhoc PUSH. (External message/API)

NDU

central-interpreter service is designed as

Stateless

no local service state per Pod

Versioned API

central-interpreter.yaml

Versioned messages

Currently, the central interpreter does not publish any messages;

While it consumes different kinds of messages such as policy/host/storage etc.

API backward/forward compatibility

- central interpreter would provide all API which is derived from policy, including
  - /v3/protection-policies
  - /v3/protection-policy-summaries
  - /v3/protection-configurations
  - /v3/protections
  - */v3/protection-details*
  - */v3/protection-metrics*

Fully data schema ownership

- N/A

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

Long Run job handling

central-interpreter service does not support any long-run job that may run across an upgrade, so it does not impact any job runs before the upgrade.

For any cache e.g.: Idempotent, performance shall be externally persisted which shall not impact the functionality.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search, CIAM

Soft dependencies

execution-planner, policy, Alert, Audit, ASC

Dependency diagram

Outgoing Dependencies

HA

central-interpreter service will be deployed as 1 instance in Santorini Wave 1.0 but may be deployed as multiple instances post Wave 1.0. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
central-interpreter	<= 40 seconds	******** Started Component central-interpreter in 39.022 seconds (JVM running for 39.111) ********
MSU (Micro Schema Upgrade)	TBD	Currently, no MSU is added for central-interpreter service.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the central-interpreter service

Shutdown time

Item	Graceful shutdown time	Comments
central-interpreter	<= 3 seconds	
Others	TBD	other overhead of Pod shutdown is out of the control of the central-interpreter service

Resilience strategy

persist idempotent key-value cache, which will be provided by API Idempotent framework - being tracked by

PPDM-225171 - Getting issue details... STATUS

Data consistency (service state persistence)

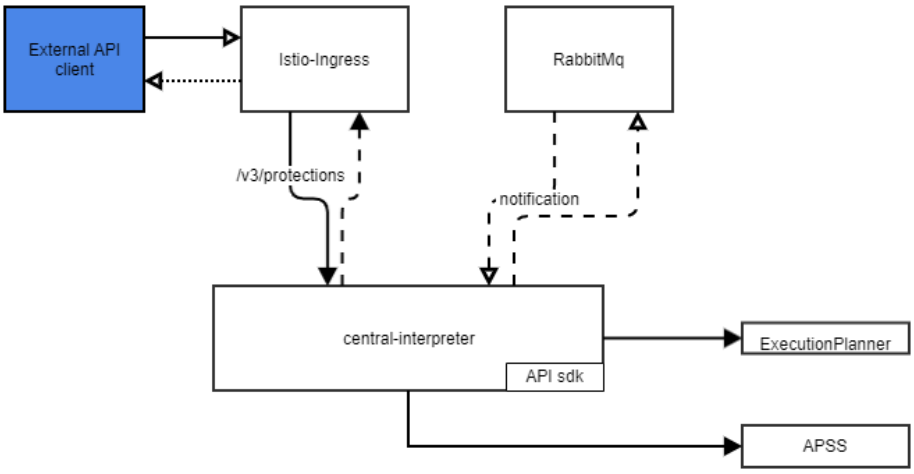
The Central Interpreter is the key component to interpret system events (including protection policy configuration etc.) to protection execution plans, which will finally drive the protection jobs execution at the scheduled pace.

The below scenarios shall be handled:

1. Keep data consistency between the execution plans for policy and policy configuration itself
2. Service restart during message handling, make sure the message is consumed successfully exactly Once
3. The retry period shall be configured properly to not be impacted by downstream service transient down

refer to Data integrity for more details

Manageability



Serviceability

Logging

central-interpreter service will follow the common logging framework to produce the service logs.

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Central Interpreter Observability

Security

PPDM RBAC

Adhoc protection is enforced with the below privileges.

Operation	Privileges	
protect now	MANAGE_POLICIES	

Refactor checklist

Execution-planner

Resource Ownership

execution-planner service is the owner of the "execution plans" resource. It provides internal REST APIs for the internal services to create execution plans to execute protection or system jobs immediately or per future schedules.

API Resources Ownership

API Type	Resource	Supported Operations
REST API	execution plans	CRUDQ (And execution callback, execute by ID)
REST API	execution plans setting	RU
REST API	execution plan instances	Q

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables
ES	index_execution_plan	NA
ES	index_execution_plan_instance	NA
ES	index_execution_plan_setting	NA

Containerization

For non-Santorini PPDM form factors, the execution planner will run in the same container with the services: policy catalog, central interpreter, and scheduler.

For the Santorini scale-out, the execution planner service runs in its owner container considering the below reasons:

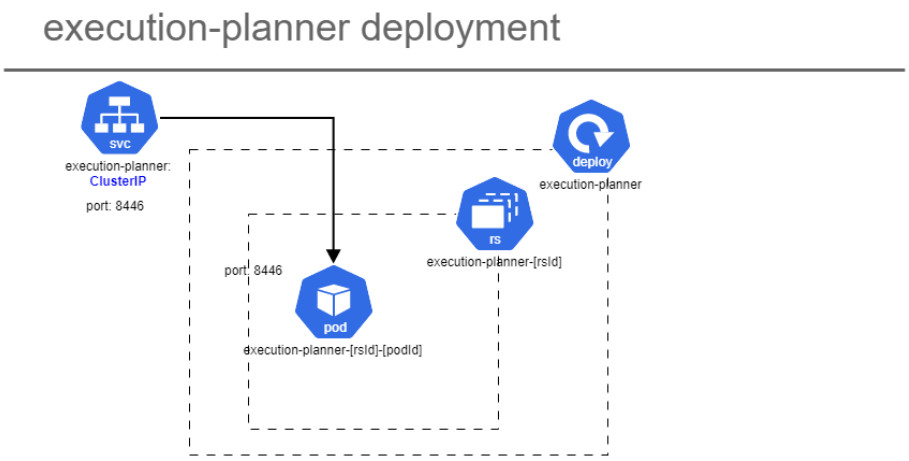
1. Not co-locate with other micro-services with more complicated service dependencies for NDU since execution planner does not depend on other PPDM services, only depends on infrastructure services: elastic search. The business depends on Job Creator service and Scheduler service.
2. and can be deployed as multiple pods to support HA in the future version

Deployment

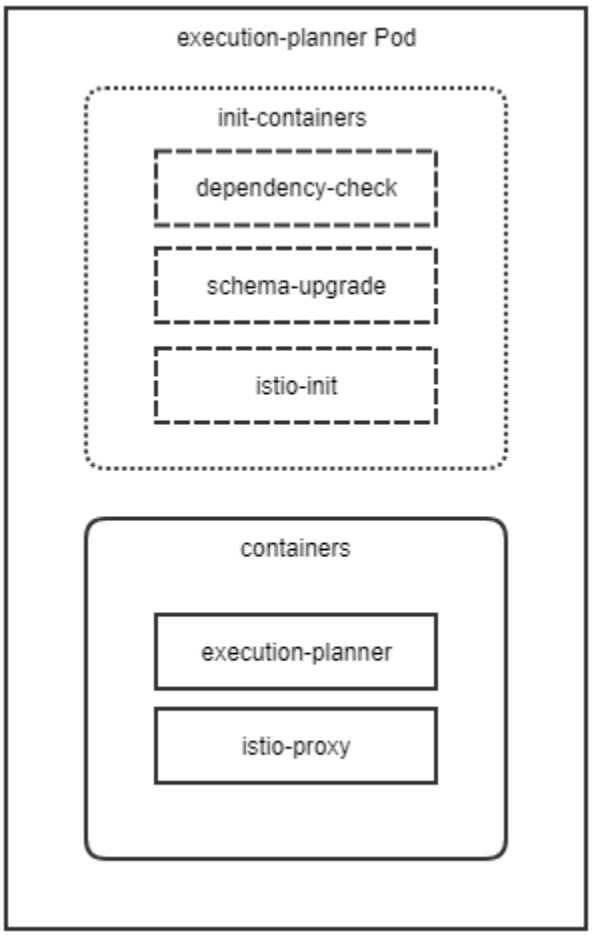
Execution planner would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

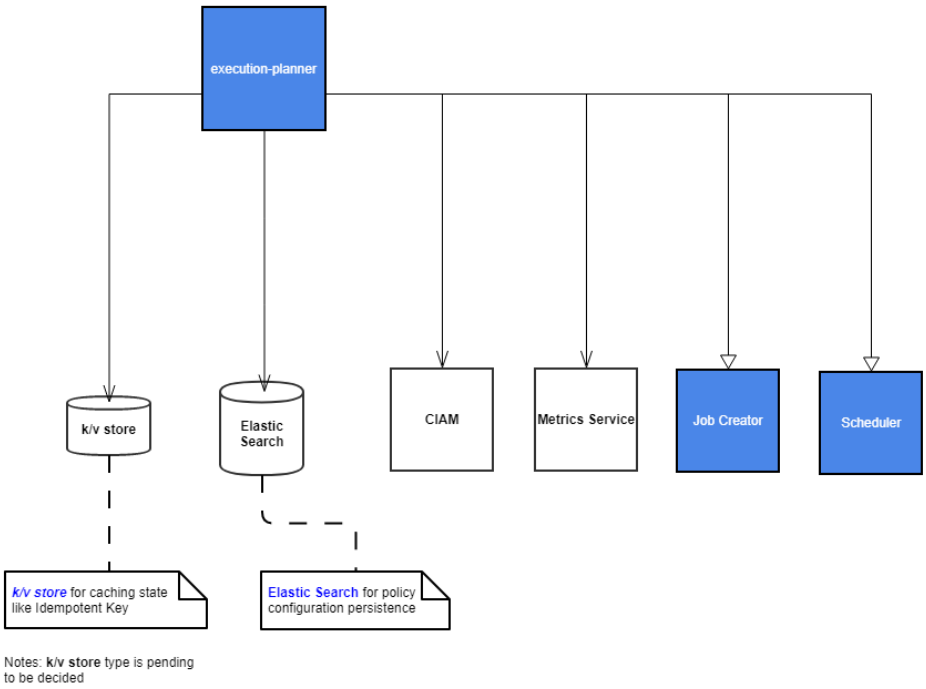
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	1G	TBD	Current image size (till R14): 620M will be based on image layering design and shared layers sizes.
Storage (App)	1.2G **Infra** / **Estimated Size** ES: index_execution_plan & index_execution_plan_instance: 1024*(1*16+47*1)*2 = 130k+20k execution plan resources (1024 policy; 48 stages per policy; 16 schedules for backup, 1 for each other stage, and double 2 to cover any ad-hoc, restore (should be clean up after 24 hours), compliance-verification job, etc, and 20k buffer) About 3k per document then the storage request for ES: 150K*4K*2(EP + EP instance, execute for 24 hours)=1200M PSQL: N/A		
Storage (Log)	100	<TODO>	in MB/day

NDU

Stateless

Execution planner is designed as no local state.
(Note: Currently there is a local scheduler trigger execution planner cleanup every day. Will need to refact that part.)

Versioned API

execution-planner.yaml

Versioned messages

NA

API backward/forward compatibility

- As a consumer service (API provider) execution-planner API supports backward compatibility
- For client service calling into execution-planner API
    - Execution-planner SDK-based Internal client service
        - For client service with older version SDK, no problem since API supports backward compatibility
        - For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade ArchitectureSpec#APIs
    - Execution-planner SDK-based API direct client service
        - For client service using older version API schema, no problem since API supports backward compatibility
        - For client service using a newer version API schema Same as SDK-based #2
    - Scheduler service will callback execution-planner using HTTP client
        - Callback does not have any dependency, but execution-planner also uses scheduler service SDK, it will follow the rule that scheduler service is as client service of execution-planner.
- Execution-planner SDK-based is a client service of CIAM, and Metric service, it shall integrate SDK of CIAM, and Metrics services. CIAM, and Metrics service shall be upgraded before the execution-planner service

Fully data schema ownership

- index_execution_plan, index_execution_plan_instance, index_execution_plan_setting
- Adopting Micro Schema Manager to schema patch for index_execution_plan, index_execution_plan_instance, index_execution_plan_setting

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

schema patch: within 10 seconds for patching execution planner data part (max: 1024 policies), not counting other overhead to launch schema patch init-container.

Long Run job handling

Execution planner service does not support any long-run job which may run across an upgrade, so it does not impact any job runs before the upgrade.

For any cache for e.g.: Idempotent, performance shall be externally persisted which shall not impact the functionality.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

Elastic Search, CIAM

Soft dependencies

Job Creator, Scheduler

Dependency diagram

Outgoing Dependencies

HA

Execution planner service will be deployed as 1 instance in Santorini Wave 1.0 but would be deployed as multiple instances post Wave 1.0 since the execution-planner service is designed as stateless. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
execution-planner	<= 30 seconds	2023-09-07T06:46:38.108Z INFO [] [main] [][][][] [c.e.d.p.e.ExecutionPlannerServiceApplication.start(296)] - ******** Started Component execution-planner in 25.239 seconds (JVM running for 25.347) ********
MSU (Micro Schema Upgrade)	<= 3 seconds	per the data migration patch, data migration chance shall be restricted MSU container may take different periods per the data to be migrated, the maximum policy configured per system is 1024, 48 stages per policy, 16 schedules for backup, total EP: 768432, adding EP instance 1572864, and there may be ad-hoc backup /restore, any other kind of request. Base on previous experience, using ES update_by_query for updating 1M docs, will cause about 5 mins. If updating doc by doc will be even slower (1 hour level). Execution planner will design avoid large data migration.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the execution-planner service

Shutdown time

Item	Graceful shutdown time	Comments

execution-planner	<= 3 seconds	2023-05-10T21:15:16.651955967-04:00 stdout F 2023-05-11T01:15:16.651Z INFO [] [Thread-36] [][][][][] [c.e.d.p.e. ExecutionPlannerServiceApplication.protectedShutdown(462)] - Shutting down.... 2023-05-10T21:15:16.652358977-04:00 stdout F 2023-05-11T01:15:16.652Z INFO [] [Thread-3] [][][][][] [c.e.b.d.b.d.e. EsTransportClientFactory.run(128)] - Closing ES client elasticsearch ... 2023-05-10T21:15:16.654631967-04:00 stdout F 2023-05-11T01:15:16.654Z INFO [] [Thread-36] [][][][][] [c.e.b.a.c.f. AccessFacade.componentLogin(245)] - ATTN: component-token: Refreshing 2023-05-10T21:15:16.703524957-04:00 stdout F 2023-05-11T01:15:16.703Z INFO [] [Thread-3] [][][][][] [c.e.b.d.b.d.e. EsTransportClientFactory.run(132)] - ES client elasticsearch closed successfully. 2023-05-10T21:15:16.72624364-04:00 stdout F 2023-05-11T01:15:16.725Z INFO [] [Thread-36] [][][][][] [c.e.b.a.c. SecurityAgentAssistant.applyAccessToken(56)] - new AccessToken has been applied. Token signature: .9xCzZuZEZ07P46g-lQu1xDdAff2KqcGzeVJ_Vz6mQDTk905NredEbzjQHxp4E7H9bJLYeWpGjMeR5Ubouge5Jw 2023-05-10T21:15:16.726909382-04:00 stdout F 2023-05-11T01:15:16.725Z INFO [] [Thread-36] [][][][][] [c.e.b.a.c.f. AccessFacade.lambda$componentLogin$16(254)] - ATTN: component-token: Successfully retrieved new component access token 2023-05-10T21:15:16.729384474-04:00 stdout F 2023-05-11T01:15:16.728Z INFO [] [Thread-36] [][][][][] [c.e.d.p.e. ExecutionPlannerServiceApplication.notifyStatusChange(521)] - Received message from sysmgr to get status 2023-05-10T21:15:16.733643816-04:00 stdout F 2023-05-11T01:15:16.732Z WARN [] [Thread-36] [][][][][] [c.e.b.p.m.u. ReflectionUtils.getMessageTypeToClassMap(101)] - Context Class Loader is not a URLClassLoader! 2023-05-10T21:15:16.740656557-04:00 stdout F 2023-05-11T01:15:16.740Z INFO [] [Thread-36] [][][][][] [c.e.d.p.e. ExecutionPlannerServiceApplication.notifyStatusChange(521)] - Received message from sysmgr to get status 2023-05-10T21:15:16.742177961-04:00 stdout F 2023-05-11T01:15:16.741Z WARN [] [Thread-36] [][][][][] [c.e.b.p.m.u. ReflectionUtils.getMessageTypeToClassMap(101)] - Context Class Loader is not a URLClassLoader! 2023-05-10T21:15:16.779744369-04:00 stdout F 2023-05-11T01:15:16.779Z INFO [] [SpringApplicationShutdownHook] [][][][][] [o.q.core.QuartzScheduler.standby(585)] - Scheduler quartzScheduler_$_NON_CLUSTERED paused. 2023-05-10T21:15:16.798258661-04:00 stdout F 2023-05-11T01:15:16.796Z INFO [] [Thread-36] [][][][][] [o.s.a.r.l. SimpleMessageListenerContainer.lambda$shutdownAndWaitOrCallback$3(652)] - Waiting for workers to finish. 2023-05-10T21:15:16.810779281-04:00 stdout F 2023-05-11T01:15:16.810Z INFO [] [SpringApplicationShutdownHook] [][][][][] [o.s.s.q.SchedulerFactoryBean.destroy(847)] - Shutting down Quartz Scheduler 2023-05-10T21:15:16.810842388-04:00 stdout F 2023-05-11T01:15:16.810Z INFO [] [SpringApplicationShutdownHook] [][][][][] [o.q.core.QuartzScheduler.shutdown(666)] - Scheduler quartzScheduler_$_NON_CLUSTERED shutting down. 2023-05-10T21:15:16.810853157-04:00 stdout F 2023-05-11T01:15:16.810Z INFO [] [SpringApplicationShutdownHook] [][][][][] [o.q.core.QuartzScheduler.standby(585)] - Scheduler quartzScheduler_$_NON_CLUSTERED paused. 2023-05-10T21:15:16.81154355-04:00 stdout F 2023-05-11T01:15:16.811Z INFO [] [SpringApplicationShutdownHook] [][][][][] [o.q.core.QuartzScheduler.shutdown(740)] - Scheduler quartzScheduler_$_NON_CLUSTERED shutdown complete. 2023-05-10T21:15:16.827645849-04:00 stdout F 2023-05-11T01:15:16.827Z INFO [] [SpringApplicationShutdownHook] [][][][][] [c.d.p.i.e.c.ConsumingEventObserverRegistry.stopConsuming(122)] - >>> stopConsuming(): in 2023-05-10T21:15:16.8279158-04:00 stdout F 2023-05-11T01:15:16.827Z INFO [] [SpringApplicationShutdownHook] [][][][][] [c.d.p.i.e.c.ConsumingEventObserverRegistry.stopConsuming(130)] - >>> stopConsuming(): out 2023-05-10T21:15:16.879425061-04:00 stdout F 2023-05-11T01:15:16.879Z INFO [] [Thread-36] [][][][][] [o.s.a.r.l. SimpleMessageListenerContainer.lambda$shutdownAndWaitOrCallback$3(656)] - Successfully waited for workers to finish. 2023-05-10T21:15:16.883706229-04:00 stdout F 2023-05-11T01:15:16.883Z INFO [] [Thread-36] [][][][][] [o.s.a.r.l. SimpleMessageListenerContainer.lambda$shutdownAndWaitOrCallback$3(652)] - Waiting for workers to finish. 2023-05-10T21:15:17.855520255-04:00 stdout F 2023-05-11T01:15:17.855Z INFO [] [Thread-36] [][][][][] [o.s.a.r.l. SimpleMessageListenerContainer.lambda$shutdownAndWaitOrCallback$3(656)] - Successfully waited for workers to finish.
Others	TBD	other overhead of Pod shutdown is out of the control of the execution-planner service

Data consistency (service state persistence)

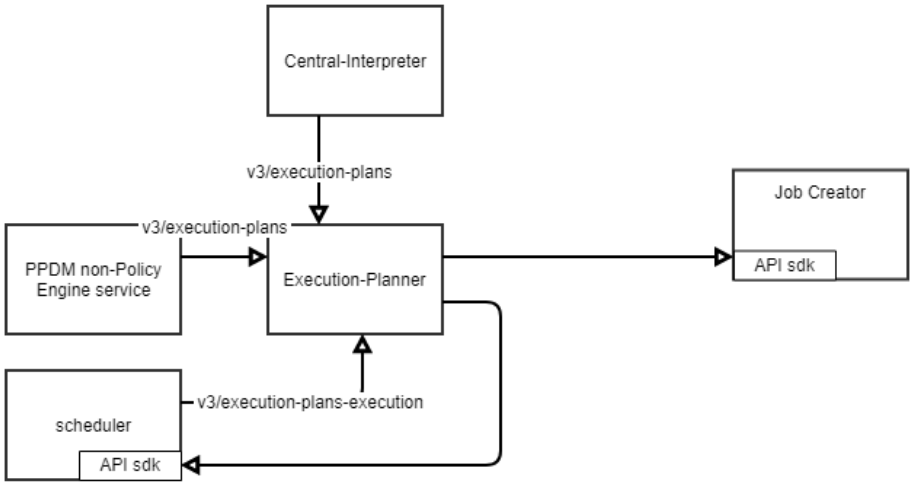
refer to 2.1. Data integrity for more details

Resilience strategy

persist idempotent key-value cache, which will be provided by API Idempotent framework - being tracked by PPDM-225171

Manageability

execution-planner service provides the Rest API for CRUD execution planners.



Serviceability

Logging

execution-planner service will follow the common logging framework to produce the service logs.

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Execution Planner Observability

Security

PPDM RBAC

Execution planner CRUD operation is enforced with the below privileges.

Operation	Privileges	
Create	MANAGE_POLICIES	
Update	MANAGE_POLICIES	
Delete	MANAGE_POLICIES	
GetById	VIEW_POLICIES	
Query	VIEW_POLICIES	

Refactor checklist

PPDM Service Refactor Checklist -Execution Planner

Scheduler

Resource Ownership

Scheduler Service is the owner of the "schedule" resource. It provides REST APIs for the other components to manage schedule and "window" on schedules.

API Resources Ownership

API Type	Resource	Supported Operations
REST API	schedules	CRUDQ

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables	Comments
ES	index_schedule_v3	index_schedule	
PSQL	*	*	ppdm_qrtz_cron_triggers like table Owned by quartz-scheduler

Containerization

For non-Santorini PPDM form factors, the scheduler will run in the same container with the services: central interpreter, execution planner, and policy-catalog.

For the Santorini scale-out, the scheduler service runs in its owner container considering the following reasons:

1. The scheduler service is based on scheduler-quartz, which is not configured as a 'Cluster' yet, so it could not be deployed as multiple Pods for now.
2. Not co-locate with other micro-services with more complicated service dependencies for NDU since scheduler does not depend on other PPDM services, only depends on infrastructure services: elastic search, rabbitMQ, CIAM, Audit log, and Alert
3. and can be deployed as multiple pods to support HA in post-Santorini Wave 1.0

API SDK

API SDK will be generated from API schema (defined by YAML) for APIs provided by the scheduler service.

scheduler API consumer shall adopt scheduler API SDK to interact with scheduler service

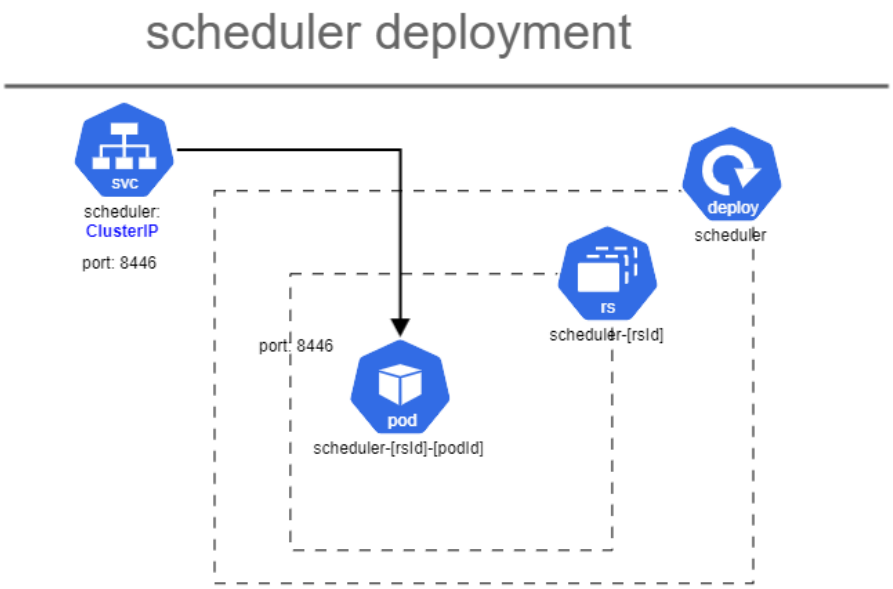
scheduler service shall adopt the API SDK of its dependent service, such as CIAM, Audit, Metrics, etc.

Deployment

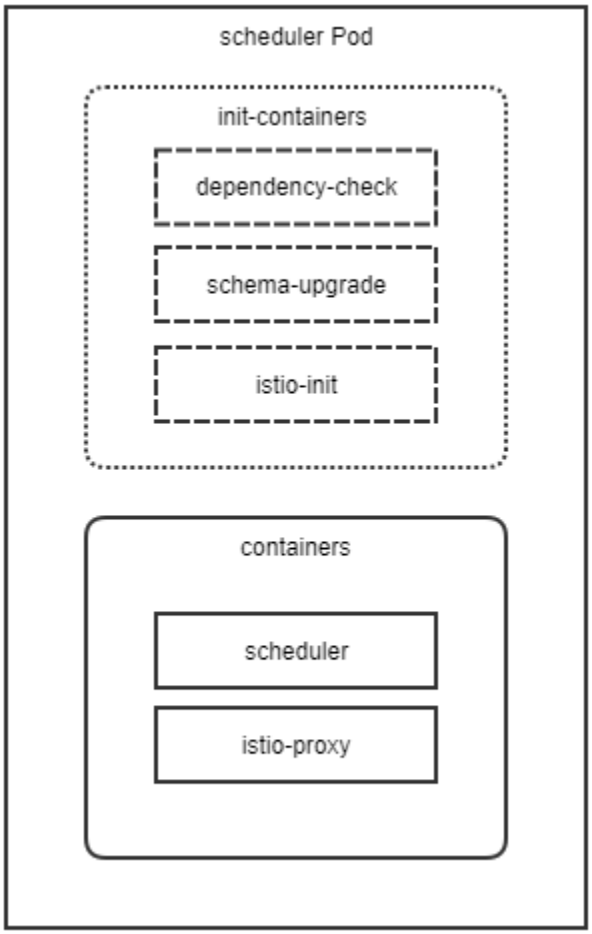
scheduler would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

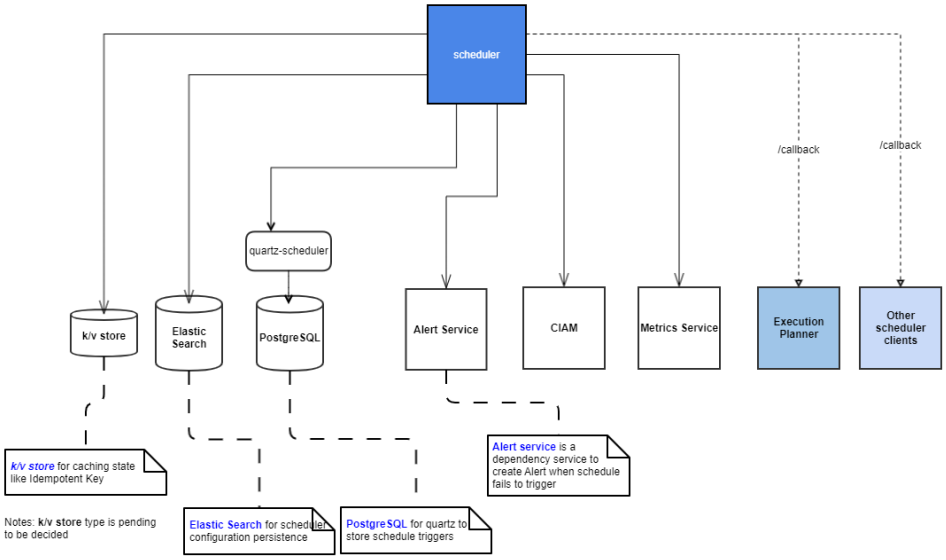
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.
Storage (App)	<table><tr><th>Infra</th><th>Estimated Size</th></tr><tr><td>ES</td><td>1024*(1*16+47*1)*2 = 130k+20k schedule resources (1024 policy; 48 stages per policy; 16 schedules for backup, 1 for each other stage, and double 2 to cover any other schedules, and 20k buffer) About 2k per document then the storage request for ES: 150K*2K=300M</td></tr><tr><td>PSQL</td><td>150K * 10k = 1500M</td></tr></table>		Zhang, Raynor - Should the Idempotent API key-value cache be counted in this part? YES: we shall estimate the potential API calls per idempotent persistent time to get the estimated storage usage.
Storage (Log)	100	<TODO>	in MB/day

NDU

Stateful

scheduler service does not have a local cache since schedule configuration and state are all persisted in the external database - ES/PSQL.

while it has a daemon service to execute job trigger in each instance, which may conflict with the daemon service in other scheduler instances.

From this end, the scheduler service is stateful, before this is addressed scheduler service will be deployed as a single Pod. Post Wave 1.0, further evaluation is needed to check how to make it stateless

(notes: The scheduler service is based on scheduler-quartz, which can be deployed in a cluster, but needs further checking whether that's enough for addressing this state in post Wave 1)

Versioned API

scheduler.yaml

Versioned messages

N/A

API backward/forward compatibility

- As a consumer service (API provider) schedule v3 API supports backward compatibility (not need compatible to schedule v2 since schedule v2 is CRS based)
- For client service calling into schedules API
  - Schedule SDK-based Internal client service
    - For client service with older version SDK, no problem since API supports backward compatibility
    - For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade Architecture Spec#APIs
  - Schedules API direct client service
    - Schedules API is designed as PPDM internal API which is not directly consumed by the PPDM customer.
    - For client service using older version API schema, no problem since API supports backward compatibility
    - For client service using a newer version API schema Same as SDK-based #2
- Scheduler is a client service of CIAM, Alert, and Metric service, it shall integrate SDK of CIAM, Audit log, and Metrics services. CIAM, Audit log, and Metrics service shall be upgraded before the Scheduler service

Fully data schema ownership

- index_schedule_v3 (notes: *index_schedule will be deprecated*)
- Adopting Micro Schema Manager to schema patch for index_schedule_v3
- PSQL:
  -

ppdm_qrtz_blob_triggers
ppdm_qrtz_calendars
ppdm_qrtz_cron_triggers
ppdm_qrtz_fired_triggers
ppdm_qrtz_job_details
ppdm_qrtz_locks
ppdm_qrtz_paused_trigger_grps
ppdm_qrtz_scheduler_state
ppdm_qrtz_simple_triggers
ppdm_qrtz_simprop_triggers
ppdm_qrtz_triggers

Data migration plan

Adopt Micro Schema Manager to do schema migration and data migration if necessary.

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

Long Run job handling

scheduler service does not support any long-run job that may run across an upgrade, so it does not impact any job runs before the upgrade.

For any cache e.g.: Idempotent, performance shall be externally persisted which shall not impact the functionality.

For any schedule misfired during the upgrade, the misfired schedule will be fired after the upgrade

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search, PostgreSQL, CIAM

Soft dependencies

execution-planner, Alert, and other schedule owner services like discovery

Dependency diagram

Outgoing Dependencies

HA

The scheduler service will be deployed as 1 instance in Santorini Wave 1.0. HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Default replica	1	TBD
Max replica	1	TBD
Scale-Out	No	TBD

Startup time

Item	Startup time	Comments
Scheduler	<= 30 seconds	
MSU (Micro Schema Upgrade)	<= 3 seconds	per the data migration patch, data migration chance shall be restricted MSU containers may take different periods per the data to be migrated.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the Scheduler service

Shutdown time

Item	Graceful shutdown time	Comments
Scheduler	<= 3 seconds	
Others	TBD	other overhead of Pod shutdown is out of the control of the Scheduler service

Resilience strategy

- Scheduler Resilience
- Schedule misfire handling Schedule Misfire Design
- Persist idempotent key-value cache, which will be provided by the API Idempotent framework

Performance implication of persisting local state externally

Idempotent key/value would be persisted in the PostgreSQL database, the additional latency of this I/O is around 50ms (per PostgreSQL Performance testing)
(provided by Idempotent API framework common code)

This will add ~50 ms latency for schedules create/update API, and API throughput will be impacted further. (Throughput and Latency will be further tested)

Data consistency (service state persistence)

refer to 1.1. Data Integrity

Manageability

Refer to Manageability in PPDM Service Refactor Checklist -Scheduler

Serviceability

Logging

scheduler service will follow the common logging framework to produce the service logs.

The scheduler service will adopt 2.1. Write to local files and use a sidecar before metrics are sent over Instrument SDK. Will further evaluate whether switching to 2.3. Write to STDOUT and use Fluentbit

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Scheduler Service Observability#SchedulerMetrics

Security

PPDM RBAC

schedules CRUD operation is enforced with the below privileges.

Operation	Privileges
Create	MANAGE_POLICIES
Update	MANAGE_POLICIES
Delete	MANAGE_POLICIES
GetById	VIEW_POLICIES
Query	VIEW_POLICIES

Refactor checklist

PPDM Service Refactor Checklist -Scheduler

Job-creator

Resource Ownership

Job creator service is the owner of the "job creation" resource. It provides APIs for the user to create jobs or retry jobs.

API Resources Ownership

API Type	Resource	Supported Operations
REST API	/api/v3/job-creation-batch	C
REST API	/api/v3/job-creator-retries	C

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables
ES	index_job_creation_context ?	NA

Containerization

For non-Santorini PPDM form factors, the job creator will run in the same container with the services: job dispatcher, job executor, and activity service.

For Santorini scale-out, job creator service runs in its owner container considering the below reasons:

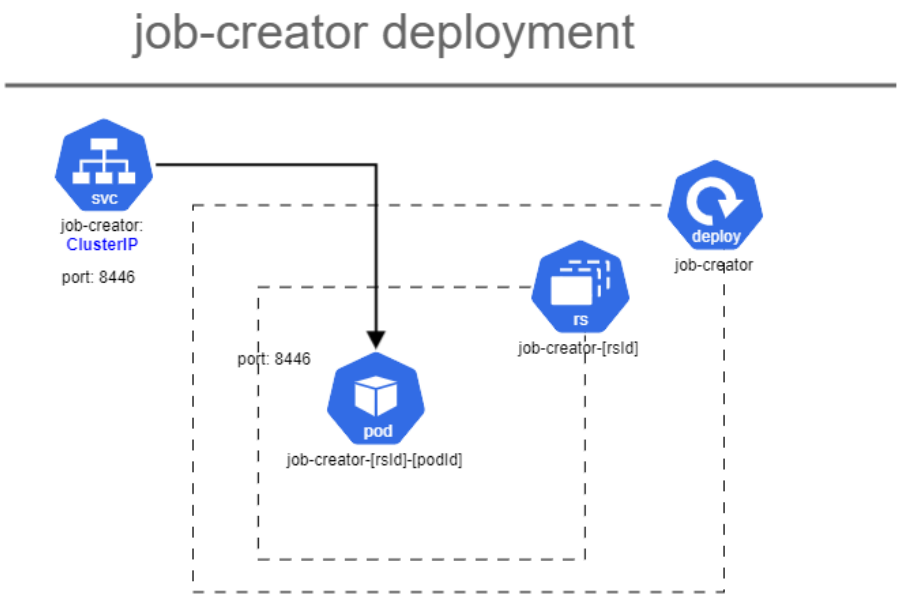
1. Not co-locate with other micro-services because job creator will depend on many other PPDM services. If only co-locate with some of them, it's hard to scale in the future.
2. It can be deployed as multiple pods to support HA in the future version.

Deployment

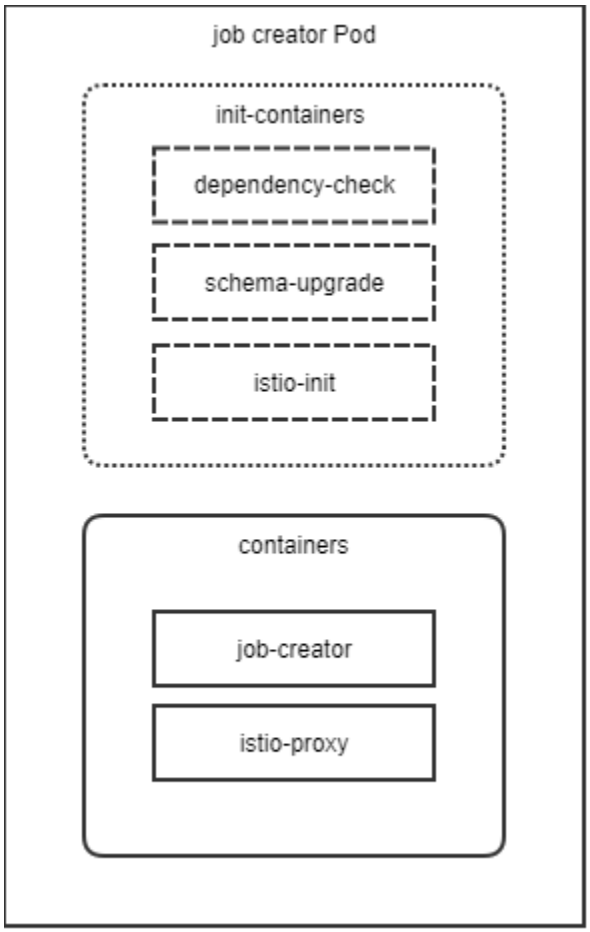
Job creator would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

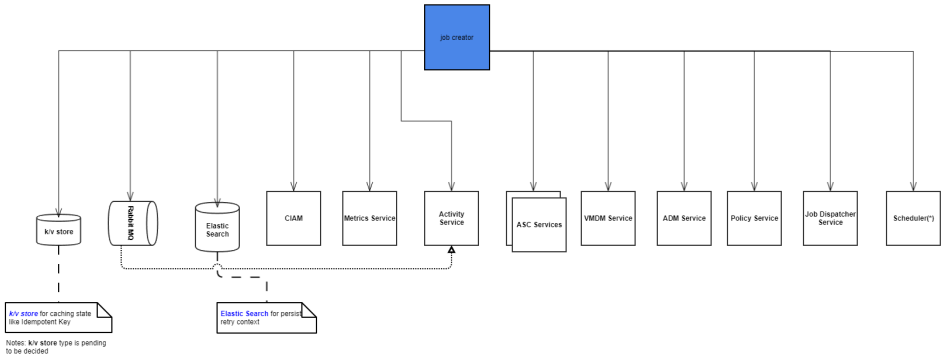
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.

Storage (App)	<table><tr><th>Infra</th><th>Estimated Size</th></tr><tr><td>ES</td><td>150k * 0.5k * 45 = 3375 M • 150k: Jobs every day • 0.5k: stage size for each job • 45: keep data for 45 days</td></tr><tr><td>PSQL</td><td>N/A</td></tr></table>		
Storage (Log)	500	<TODO>	in MB/day

NDU

Job creator service is designed as no local state per pod, however, it does have externalized state that is consumed by the service, additional handling is needed when multiple instances will be supported post Santorini Wave 1.0

Versioned API

job-creator.yaml

- /v3/job-creation-batch
- /v3/job-creator-retries

Versioned messages

N/A

job-creator does not publish any message over the message bus;

while it does consume messages - activity retry/cancel/status change over the message bus.

API backward/forward compatibility

- As a consumer service (API provider) job-creator API supports backward compatibility
- For client service calling into job creator API
    - SDK-based Internal client service
        - For client service with older version SDK, no problem since API supports backward compatibility
        - For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade Architecture Spec#APIs

Fully data schema ownership

- index_job_intervention_context
- index_job_creation_request
- Adopting Micro Schema Manager to schema patch for index_job_intervention_context and index_job_creation_request

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

schema patch: within 30 seconds for patching index_job_creation_context data, not counting other overhead to launch schema patch init-container.

Don't patch all the old data. Just some of them meet a certain condition.

Long Run job handling

job-creator may take up to several minutes to create jobs for running a policy protecting 1000+ assets and continue to resume the job creation if it's stopped by system upgrade. By the way, this is not specific to system update, also applicable for service restart in HA events.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search, CIAM

Soft dependencies

policy, job dispatcher, activity, VMDM, ADM, ASC, Alert

Dependency diagram

Outgoing dependencies

HA

job-creator service will be deployed as 1 instance in Santorini Wave 1.0 but may be deployed as multiple instances post Wave 1.0. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
Job Creator	<= 30 seconds	
MSU (Micro Schema Upgrade)	<= 3 seconds	per the data migration patch, data migration chance shall be restricted MSU containers may take different periods per the data to be migrated.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the Job Creator service

Shutdown time

Item	Graceful shutdown time	Comments
Job Creator	<= 3 seconds	
Others	TBD	Other overhead of Pod shutdown is out of the control of the Scheduler service

Resilience strategy

persist idempotent key-value cache, which will be provided by API Idempotent framework - being tracked by PPDM-225171

Data consistency (service state persistence)

Refer to Job Creator for more details

Manageability

Serviceability

Logging

job creator service will follow the common logging framework to produce the service logs.

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Job Creator Observability - eCDM - ISG Confluence (internal) (emc.com)

Security

Don't need RBAC, or IBAC because it's internal API and always trust the request from upstream components.

Refactor checklist

PPDM Service Refactor Checklist -Job Creator

Job-dispatcher

Resource Ownership

API Resources Ownership

API Type	Resource	Supported Operations
REST API	/api/v3/job-dispatch	C
REST API	/api/v3/job-dispatch/{jobId}/cancellation	C

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables
ES	index_queued_workflow_execution	NA

Containerization

For non-Santorini PPDM form factors, the job dispatcher will run in the same container with the services: job creator, job executor, and activity service.

For Santorini scale-out, job dispatcher service runs in its owner container considering the below reasons:

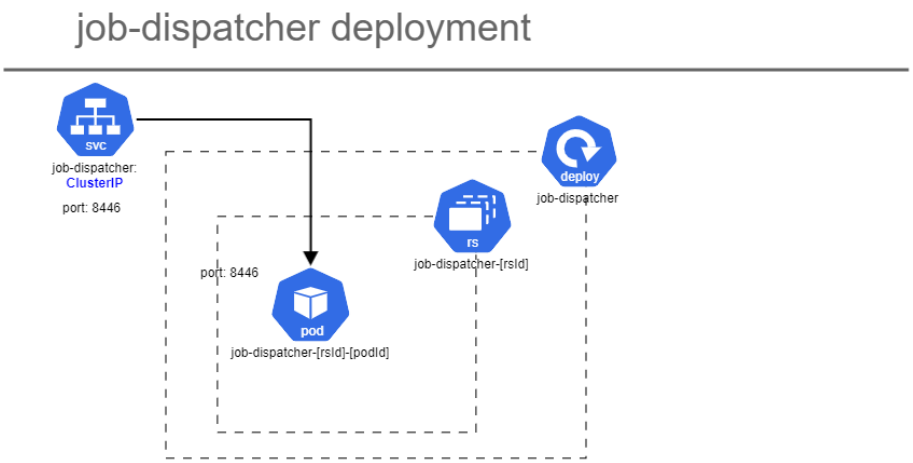
1. Not co-locate with other micro-services with more complicated service dependencies for NDU since the job dispatcher does not depend on other PPDM services, only depends on infrastructure services and job executor.
2. It can be deployed as multiple pods to support HA in the future version.

Deployment

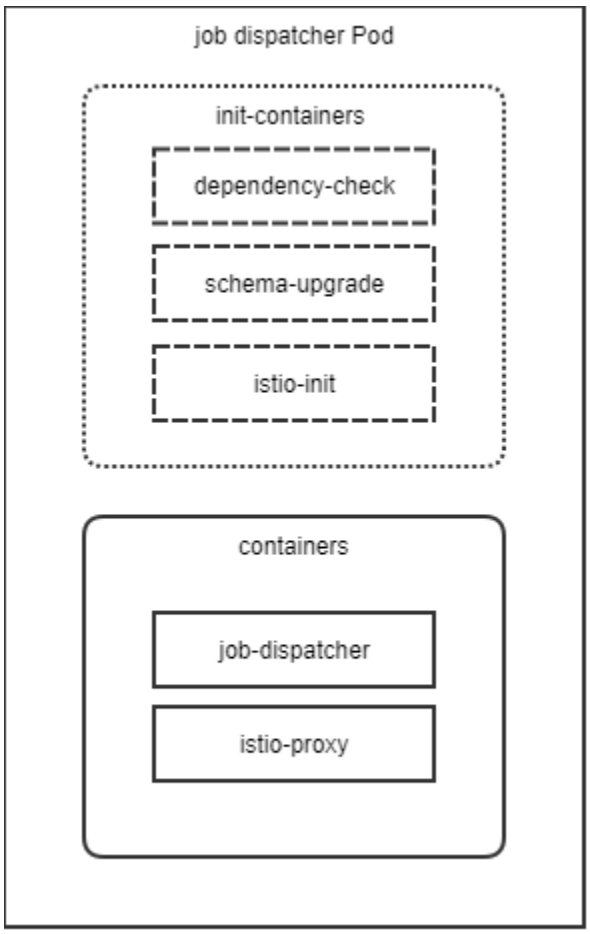
Job dispatcher would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

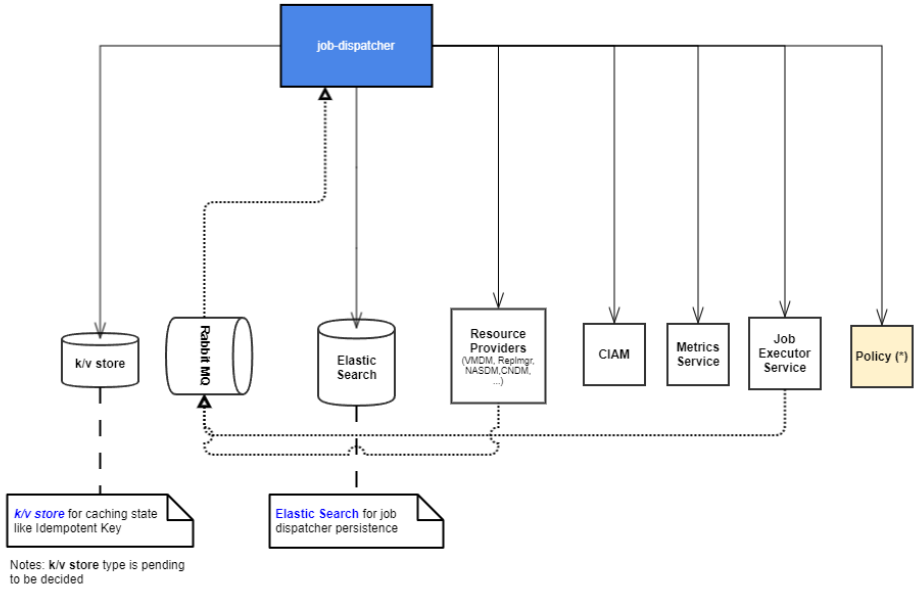
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.
Storage (App)	<table><tr><th>Infra</th><th>Estimated Size</th></tr><tr><td>ES</td><td>15k * 1k = 15 M • 15k: max queued jobs • 1k: storage size per job</td></tr><tr><td>PSQL</td><td>N/A</td></tr></table>		
Storage (Log)	1000	<TODO>	in MB/day

NDU

Job-dispatcher service is stateful since it includes daemon threads to dispatch jobs from the job queue, threads in multiple job dispatcher instances may conflict with dispatching the same job. On the other hand resource bookkeeping, and job conflict management cache shall be well managed across multiple instances.

Versioned API

job-dispatcher.yaml

Versioned messages

NA
(Job dispatcher does not produce messages, but it consumes messages such as resource usage update messages, etc. delivered by job-executor, pillars such as NAS)

API backward/forward compatibility

- As a consumer service (API provider) job-dispatcher API supports backward compatibility
- For client service calling into job-dispatcher API
    - SDK-based Internal client service
        - For client service with older version SDK, no problem since API supports backward compatibility

- For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade Architecture Spec#APIs

Fully data schema ownership

- index_job_resource_usage
- index_queued_workflow_execution
- Adopting Micro Schema Manager to schema patch

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

schema patch: within 120 seconds for patching job dispatcher data part (max: 100k jobs), not counting other overhead to launch schema patch init-container.

Long Run job handling

The queued jobs will be resumed after the upgrade.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search, CIAM

Soft dependencies

job-executor, VMDM, ReplMgr

Dependency diagram

Outgoing Dependencies

HA

Job dispatcher service will be deployed as 1 instance in Santorini Wave 1.0 but would be deployed as multiple instances post Wave 1.0. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
Job dispatcher	<= 30 seconds	
MSU (Micro Schema Upgrade)	<= 3 seconds	per the data migration patch, data migration chance shall be restricted MSU containers may take different periods per the data to be migrated.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the Job dispatcher service

Shutdown time

Item	Graceful shutdown time	Comments
Job dispatcher	<= 3 seconds	
Others	TBD	other overhead of Pod shutdown is out of the control of the Job dispatcher service

Resilience strategy

persist idempotent key-value cache, which will be provided by API Idempotent framework - being tracked by PPDM-225171

Data consistency (service state persistence)

refer to Job Dispatcher for more details

Manageability

Serviceability

Logging

job dispatcher service will follow the common logging framework to produce the service logs.

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Job Dispatcher Observability - eCDM - ISG Confluence (internal) (emc.com)

Security

Don't need RBAC, or IBAC because it's internal API and always trust the request from upstream components.

Refactor checklist

PPDM Service Refactor Checklist -Job Dispatcher

Job-executor

Resource Ownership

API Resources Ownership

API Type	Resource	Supported Operations
REST API	JobExecution	CQ

Storage Index (ES/PSQL) Ownership

Database	Finalized Index/Table	Original Indexes/Tables
PSQL	JOB_EXECUTION	NA
PSQL	ACTION_RESULT	NA

Containerization

For non-Santorini PPDM form factors, the job executor will run in the same container with the services: job creator, job executor, and activity service.

For Santorini scale-out, job executor service runs in its owner container considering the below reasons:

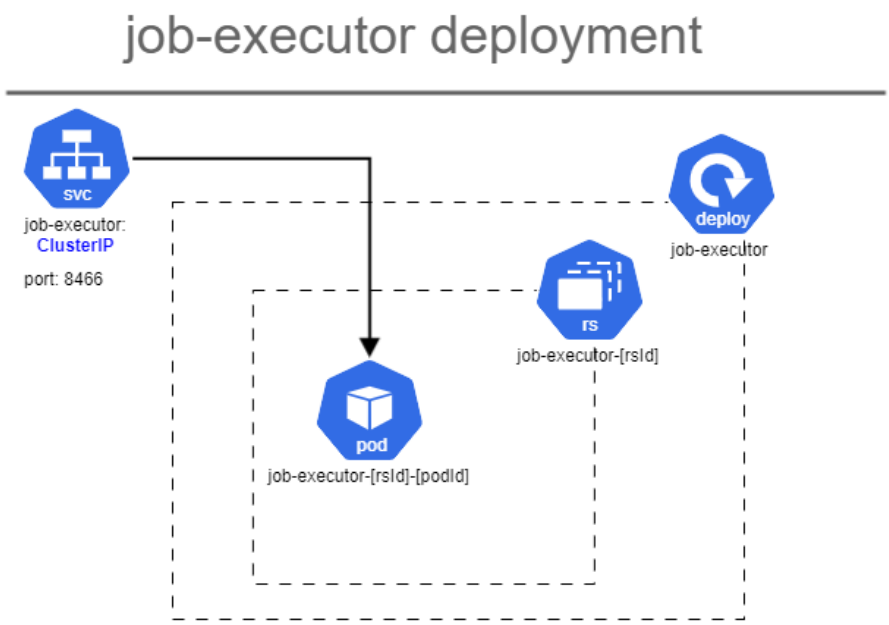
1. Not co-locate with other micro-services with more complicated service dependencies for NDU since job executor does not depend on other PPDM services, only depends on infrastructure services.
2. It can be deployed as multiple pods to support HA in the future version.

Deployment

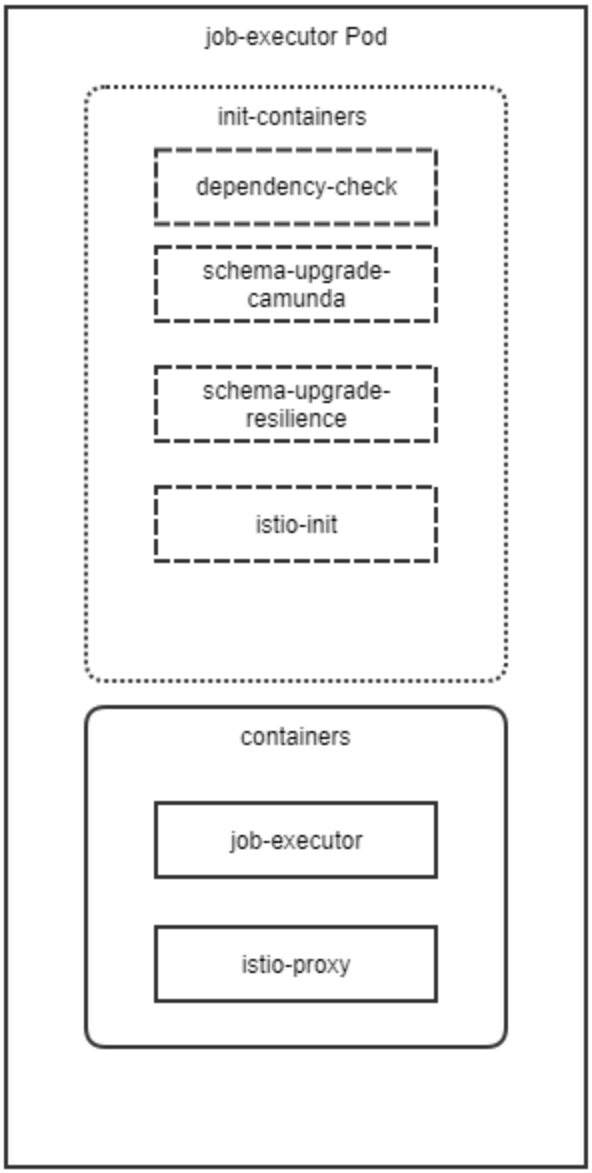
Job executor would be deployed as a 'Deployment' resource type in Santorini, the deployment layout is as below.

k8s resources

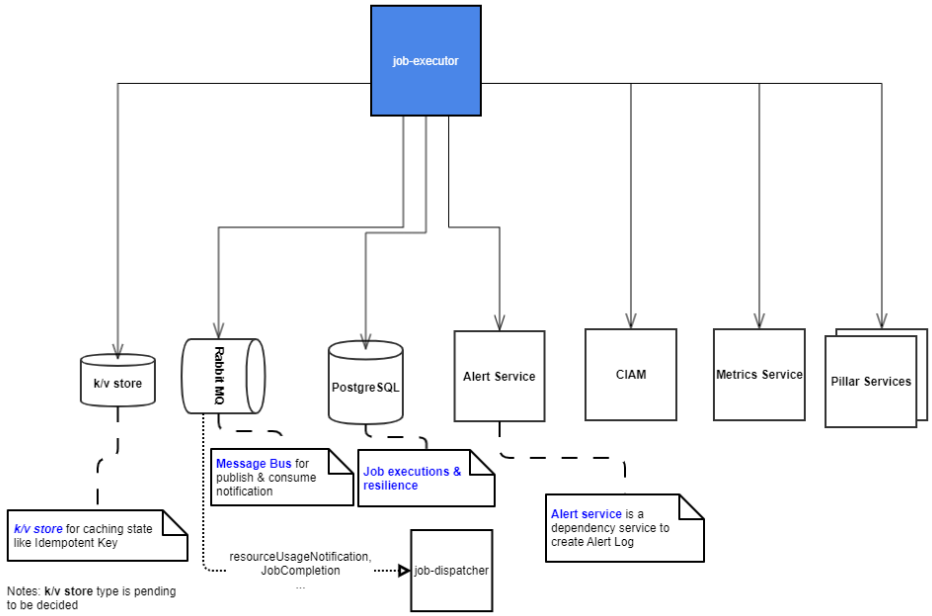
- Deploy Service/Deployment/Pod



Pod internals



Outgoing dependencies



Resources

Item	Requested Size (Min)	Limited Size (Max)	Comments
CPU	500m = 0.5 CPU	2 Virtual CPUs	refer to assign-cpu-resource Need further test what's the performance within these CPU resource constraints.
Memory	512MB	768MB	refer to assign-memory-resource
Disk (Image Size)	TBD	TBD	will be based on image layering design and shared layers sizes.
Storage (App)	<table><tr><th>Infra</th><th>Estimated Size</th></tr><tr><td>ES</td><td>1024 * 5k *48 = 240Mi * 2 = 500Mi • 1024: max policy number per PPDM system • 5k: stage size with 16 schedules /retentions • 48: maximum stage size in a policy</td></tr><tr><td>PSQL</td><td>N/A</td></tr></table>		
Storage (Log)	1000	<TODO>	in MB/day

NDU

job-executor service is designed as stateless.

Versioned API

job-executor.yaml

Versioned messages

- resourceUsageNotification
- JobCompletion

API backward/forward compatibility

- As a consumer service (API provider) job-executor API supports backward compatibility
- For client service calling into job-executor API
  - SDK-based Internal client service
    - For client service with older version SDK, no problem since API supports backward compatibility

- For client service with a newer versioned SDK, since strict API validation is enforced, then the CU (create/update) request may fail since the old version API does not know some new fields supported in the new version API. Even if strict API validation is not enforced, new information provided by the client may be missed by the API provider. This should be handled by proper dependency planning per PPDM in Santorini Non-Disruptive Upgrade Architecture Spec#APIs

Fully data schema ownership

Use PostgreSQL as the database to store below information:

- job-executor.job_execution
- job-executor.http_request_retry
- job-executor.action_result
- job-executor.flyway_schema_history
- and tables being used by workflow engine - Camunda.

Upgrade type

Recreate

Upgrade time range

shutdown: within 5 seconds

startup: within 30 seconds

Long Run job handling

The queued/running jobs will be resumed after the upgrade.

Dependencies

Hard dependencies

(Must start before this microservice, Usually included in Initcontainers)

RabbitMQ, Elastic Search,PostgreSQL, CIAM

Soft dependencies

policy, activity, pillar services (VMDM, ADM, NASDM, Replmgr,...), ASC, Alert

Dependency diagram

Outgoing Dependencies

HA

Job-executor service will be deployed as 1 instance in Santorini Wave 1.0 but would be deployed as multiple instances post Wave 1.0 since the job-executor service is designed as stateless. In Wave 1.0 HA will be supported by service fast reboot/startup plus SDK retry within a period longer than the service restart period.

Item	Wave 1.0	Post Wave 1.0
Scale-Out	No	Yes
Default replica	1	TBD
Max replica	1	TBD

Startup time

Item	Startup time	Comments
job-executor	<= 30 seconds	
MSU (Micro Schema Upgrade)	TBD	Currently, there's no MSU for job-executor, but in WAVE 1.0, we need to add **MSU of PSQL** databases. There are 2 PSQL databases in job-executor that need to be upgraded. 1. The camunda database a. For camunda version upgrade, the time cost is steady and usually about **1~2 second** for each patch.

		b. For process instance data migration, there is no data in Wave 1.0, each patch is usually about **0.5 seconds** 2. The job-executor resilience database a. Since no data in Santorini Wave 1.0, each patch is usually about **0.5 seconds**.
Others	TBD	dependency-check, istio-init, istio-proxy Start time consumed by other containers such as dependency-check, istio-init, and istio-proxy are out of the control of the Job executor service

Shutdown time

Item	Graceful shutdown time	Comments
job-executor	<= 3 seconds	
Others	TBD	other overhead of Pod shutdown is out of the control of the Job executor service

Data consistency (service state persistence)

Refer to 2.3. Job Executor fore more details

Manageability

Serviceability

Logging

job executor service will follow the common logging framework to produce the service logs.

See: Log Persistency and Log Bundle Creation in Santorini - eCDM - ISG Confluence (internal) (emc.com)

Metrics

API latency/throughput metrics would be sent to the metrics service over Instrument SDK, refer to Observability for PPDM - eCDM - ISG Confluence (internal) (emc.com) for more details.

For observability, these metrics would be visualized in the metrics dashboard (Grafana)

more details: Job Executor Observability

Security

Refactor checklist

PPDM Service Refactor Checklist -Job Executor

Activity

tracked by another design EPIC PPDM-225205